

1. Objectives:

1. To understand the fundamental concepts of searching algorithms in Data Structures.
2. To understand how Linear Search algorithm works and its limitations.
3. To understand how Binary Search algorithm works and its advantages.

2. Introduction:

- **Searching:** Searching is one of the most fundamental operations in data structures and algorithms. It refers to the process of locating a specific element within a collection of data. Searching plays an important role in information retrieval, database management, memory access, and real-time systems. There are different searching techniques based on the structure and organization of data. Among them, Linear Search and Binary Search are two basic and widely used methods.
- **Linear Search:** Linear Search is the simplest searching technique. In this method, each element of the array or vector is checked sequentially from the beginning until the desired element is found or the end of the list is reached. It does not require the data to be sorted, which makes it flexible but less efficient for large datasets. The time complexity of Linear Search is $O(n)$, where n is the number of elements. We iterate over all the elements of the array and check if the current element is equal to the target element. If we find any element to be equal to the target element, then return the index of the current element. Otherwise, if no element is equal to the target element, then return -1 as the element is not found. Linear search is also known as sequential search.

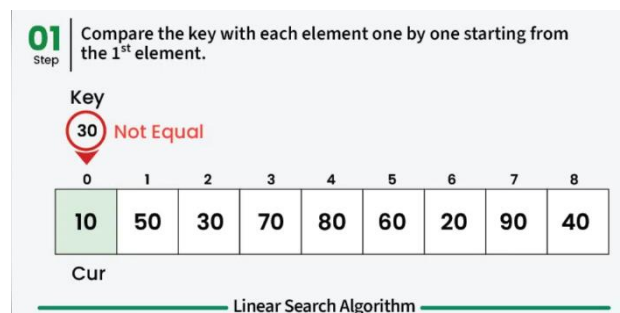


Fig.3.1: Linear Search

- **Binary search:** Binary Search, on the other hand, is a more efficient searching technique that works only on sorted data. It repeatedly divides the search space into two halves by comparing the target value with the middle element. If the value matches, the search ends; otherwise, the search continues in the appropriate half. This strategy makes Binary Search significantly faster, with a time complexity of $O(\log n)$. However, it requires the data to be sorted before execution.

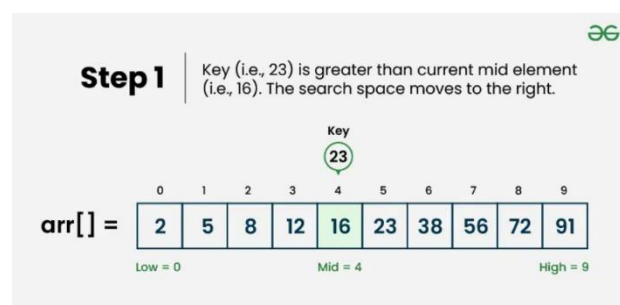


Fig.3.2: Binary Search

3. Procedure

Experiment 1: Linear Search

Input:

- $v[]$ – A vector containing integer elements
- n – Size of the vector
- q – Query element

Output:

- i – Index of the value if found
- 0 – If value is not found

Algorithm:

1. $i \leftarrow 0$
2. **while** $i < n$ **do**
3. **if** $v[i] == q$ **then**
4. **print** "Found"
5. **return** i
6. $i \leftarrow i + 1$
7. **print** "Not Found"
8. **return** 0

Experiment 2: Find duplicate value using Linear Search

Input:

- $v[]$ – A sorted vector containing integer elements
- n – Size of the vector

Output:

- 1 – If duplicate is found
- 0 – If no duplicate is found

Algorithm:

1. $i \leftarrow 0$
2. **while** $i < n$ **do**
3. **if** $v[i] == v[i + 1]$ **then**
4. **print** "Duplicate Found"
5. **return** i
6. $i \leftarrow i + 1$
7. **print** "Duplicate NOT FOUND"
8. **return** 0

Experiment 3: Binary Search

Input:

- $v[]$ – A sorted vector containing integer elements
- n – Size of the vector
- q – Query element

Output:

- i – Index of the value if found
- 0 – If the value is not found

Algorithm:

1. $start \leftarrow 0$
2. $last \leftarrow n - 1$
3. **while** $start \leq last$ **do**
4. $mid \leftarrow (start + last) / 2$
5. **if** $v[mid] == q$ **then**
6. **print** "Found"
7. **return** mid
8. **else if** $q > v[mid]$ **then**
9. $start \leftarrow mid + 1$
10. **else**
11. $last \leftarrow mid - 1$
12. **print** "Not Found"
13. **return** 0

4. Required Hardware and Software:

- **Hardware:** AMD RYZEN 7, RAM 8 GB
- **Software:** Visual Studio Code 1.109

5. Experiment Implementation:

Experiment 1: Linear Search

```
#include <bits/stdc++.h>
using namespace std;

int main(){
    int n;
    cout << "Input Size: ";
    cin >> n;

    vector<int> v;
    for (int i = 0; i < n; i++){
        int k;
        cin >> k;
        v.push_back(k);
    }

    int q;
    cout << "Input Query: ";
    cin >> q;

    for (int i = 0; i < n; i++){
        if (v[i] == q){
            cout << "Found" << endl;
            return 0;
        }
    }
}
```

```

    }
}

cout << "Not Found" << endl;
return 0;
}

```

Table 3.1: Experiment 1 output

Input	Expected Output	Actual Output
Input Size: 5 Input Elements: 1 3 5 7 9 Input Query: 5	Input Size: 5 Input Elements: 1 3 5 7 9 Input Query: 5 Found	Input Size: 5 Input Elements: 1 3 5 7 9 Input Query: 5 Found

Experiment 2: Find double value using Linear Search

```

#include <bits/stdc++.h>
using namespace std;

int main(){
    vector<int> v = {1, 2, 3, 3, 5};

    for (int i = 0; i < v.size() - 1; i++){
        if (v[i] == v[i + 1]){
            cout << "Duplicate found" << endl;
            return 0;
        }
    }

    cout << "Duplicate NOT FOUND";

    return 0;
}

```

Table 3.2: Experiment 2 output

Input	Expected Output	Actual Output
	Duplicate found	Duplicate found

Experiment 3: Binary Search

```

#include <bits/stdc++.h>
using namespace std;

int main(){
    int n;
    cout << "Input Size: ";
    cin >> n;
    cout << "Input Elements: ";

    vector<int> v;
    for (int i = 0; i < n; i++){
        int k;
        cin >> k;
        v.push_back(k);
    }

    int q;
    cout << "Input Query: ";
    cin >> q;

    int start = 0;
    int last = n - 1;

    while (start <= last){

```

```

        int mid = (start + last) / 2
        if (v[mid] == q){
            cout << "Found" << endl;
            return 0;
        }
        else if (q > v[mid]){
            start = mid + 1;
        }
        else{
            last = mid - 1;
        }
    }

    cout << "Not Found" << endl;
    return 0;
}

```

Table 3.3: Experiment 3 output

Input	Expected Output	Actual Output
Input Size: 5 Input Elements: 2 4 6 8 10 Input Query: 10	Input Size: 5 Input Elements: 2 4 6 8 10 Input Query: 10 Found	Input Size: 5 Input Elements: 2 4 6 8 10 Input Query: 10 Found

6. Discussion & Conclusion:

In this lab, we have learned the core concept of searching in data structures by implementing Linear Search and Binary Search. Linear Search is simple and does not require sorted data but becomes inefficient as the dataset grows larger. On the other hand, Binary Search is significantly better due to its divide-and-conquer approach, but it requires sorted input data. While implementing these algorithms, we faced some indexing and syntax errors in loop conditions, and boundary values. Through practical implementation, we gained a clear understanding of how different searching techniques operate, their advantages and limitations, and when each method should be used. Overall, this experiment strengthened our understanding of searching algorithms and their importance.

7. Reference:

1. <https://www.geeksforgeeks.org/dsa/linear-search/> accessed at 6.25 PM on 19th February 2026.
2. <https://www.geeksforgeeks.org/dsa/binary-search/> accessed at 7.35 PM on 20th February 2026.